

Supervised Learning — Quick Reference

Models, multiclass strategies, and utility functions

One vs One Classifier

Multiclass strategy

PROCESS

Trains one classifier for each pair of classes. Each classifier votes, and the class with the most votes wins.

HYPERPARAMETERS

estimator The base classifier to use, e.g. `LogisticRegression()`

PROS / CONS

- ✓ Works well for small datasets.
- ✗ Gets slow quickly — needs k^2 classifiers for k classes.

COMMON USES — Multiclass problems with few classes.

CODE

```
from sklearn.multiclass import OneVsOneClassifier
from sklearn.linear_model import LogisticRegression

model = OneVsOneClassifier(LogisticRegression())
model.fit(X_train, y_train)
```

One vs All Classifier

Multiclass strategy

PROCESS

Trains one classifier per class (this class vs. everything else). The class with the highest score wins.

HYPERPARAMETERS

estimator The base classifier to use, e.g. `LogisticRegression()`

PROS / CONS

- ✓ Simpler and faster than One vs One.
- ✗ Struggles when classes are very unbalanced.

COMMON USES — Image and text classification.

CODE

```
from sklearn.multiclass import OneVsRestClassifier
from sklearn.linear_model import LogisticRegression

model = OneVsRestClassifier(LogisticRegression())
model.fit(X_train, y_train)
```

Decision Tree Classifier

Rule-based tree

PROCESS

Splits data into groups by asking yes/no questions about features. Each split tries to keep the resulting groups as pure as possible.

HYPERPARAMETERS

- `max_depth` How many levels deep the tree can go
- `min_samples_leaf` Minimum samples needed to form a leaf

PROS / CONS

- ✓ Easy to read and explain; no need to scale features.
- ✗ Overfits easily — always set `max_depth`.

COMMON USES — Credit scoring, medical diagnosis.

CODE

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5)
model.fit(X_train, y_train)
```

Decision Tree Regressor

Rule-based tree

PROCESS

Same as the classifier, but instead of a class label each leaf returns the average value of its training samples.

HYPERPARAMETERS

- `max_depth` How many levels deep the tree can go
- `min_samples_leaf` Minimum samples needed to form a leaf

PROS / CONS

- ✓ Easy to interpret; handles nonlinear data.
- ✗ Overfits easily; sensitive to noisy data.

COMMON USES — House price prediction, demand forecasting.

CODE

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=5)
model.fit(X_train, y_train)
```

Linear SVM Classifier

Maximum margin classifier

PROCESS

Draws a line (or plane) between classes with the widest possible gap on either side. Kernels let it handle curved boundaries.

HYPERPARAMETERS

- C** How strict the boundary is — lower = more tolerant of errors
- kernel** Shape of the boundary: 'linear', 'rbf', 'poly'
- gamma** How far each point's influence reaches (rbf/poly only)

PROS / CONS

- ✓ Works well with lots of features.
- ✗ Slow on large datasets; features need to be scaled.

COMMON USES — Text and image classification.

CODE

```
from sklearn.svm import SVC

model = SVC(kernel='linear', C=1.0)
model.fit(X_train, y_train)
```

K-Nearest Neighbors Classifier

Instance-based learner

PROCESS

Finds the k most similar training points and returns their majority class. No training needed — it just memorises the data.

HYPERPARAMETERS

- n_neighbors** How many neighbours to look at
- weights** 'uniform' = equal vote, 'distance' = closer = more weight
- algorithm** How to search for neighbours: 'auto', 'kd_tree', 'brute'

PROS / CONS

- ✓ Simple and needs no training step.
- ✗ Slow to predict on large data; scale your features first.

COMMON USES — Recommendations, image recognition.

CODE

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5, weights='uniform')
model.fit(X_train, y_train)
```

Random Forest Regressor

Ensemble – parallel trees

PROCESS

Trains many decision trees on random subsets of the data, then averages their predictions. The randomness makes it much harder to overfit.

HYPERPARAMETERS

n_estimators Number of trees — more = better, but slower

max_depth Max depth of each tree

PROS / CONS

- ✓ Much less likely to overfit than a single tree.
- ✗ Uses more memory and takes longer to train as trees increase.

COMMON USES — Sales and price forecasting.

CODE

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(n_estimators=100, max_depth=5)
model.fit(X_train, y_train)
```

XGBoost Regressor

Ensemble – sequential trees

PROCESS

Builds trees one by one, where each new tree fixes the mistakes of the previous ones. Often the most accurate option for structured data.

HYPERPARAMETERS

n_estimators Number of trees to build

learning_rate How much each tree contributes — smaller = more careful

max_depth Max depth of each tree

PROS / CONS

- ✓ Very accurate; handles missing values automatically.
- ✗ Slow to train; many settings to tune.

COMMON USES — Competition ML (Kaggle), any structured data problem.

CODE

```
import xgboost as xgb

model = xgb.XGBRegressor(
    n_estimators=100, learning_rate=0.1, max_depth=5)
model.fit(X_train, y_train)
```

OneHotEncoder

Categorical encoding

PROCESS

Turns a categorical column into one binary column per category. E.g. 'Red/Green/Blue' becomes three 0/1 columns.

PROS / CONS

COMMON USES — Encoding categorical input features before model training.

CODE

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse=False)
encoded_data = encoder.fit_transform(categorical_data)
```

accuracy_score

Evaluation metric

PROCESS

What percentage of predictions were correct. Simple, but misleading when one class is much more common than others.

PROS / CONS

COMMON USES — Quick check on balanced classification results.

CODE

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_true, y_pred)
```

LabelEncoder

Categorical encoding

PROCESS

Converts string class labels to numbers (alphabetical order). Only use on the target column, not on input features.

PROS / CONS

COMMON USES — Encoding the target (y) column before training.

CODE

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
encoded_labels = encoder.fit_transform(labels)
```

plot_tree

Visualisation

PROCESS

Draws a decision tree as a diagram so you can see exactly which questions it asks at each step.

PROS / CONS

COMMON USES — Checking and explaining a decision tree model.

CODE

```
from sklearn.tree import plot_tree

plot_tree(model, max_depth=3, filled=True)
```

normalize

Numerical scaling

PROCESS

Scales each row to length 1. Different from StandardScaler which works column by column — use this when the direction of a row matters more than its size.

PROS / CONS

COMMON USES — Text vectors, any model that uses cosine similarity.

CODE

```
from sklearn.preprocessing import normalize

normalized_data = normalize(data, norm='l2')
```

compute_sample_weight

Class imbalance

PROCESS

Gives higher weight to samples from rare classes so the model doesn't ignore them. Pass the result into model.fit().

PROS / CONS

COMMON USES — Training on datasets where some classes have far fewer examples.

CODE

```
from sklearn.utils.class_weight import compute_sample_weight

weights = compute_sample_weight(class_weight='balanced', y=y)
model.fit(X_train, y_train, sample_weight=weights)
```

roc_auc_score

Evaluation metric

PROCESS

Measures how well a classifier separates the two classes, regardless of where you set the decision threshold. 1.0 = perfect, 0.5 = random.

PROS / CONS

COMMON USES — Comparing classifiers, especially when classes are unbalanced.

CODE

```
from sklearn.metrics import roc_auc_score

auc = roc_auc_score(y_true, y_score)
```